

Sistemas operativos

Tema 3: Estructura del sistema operativo



1

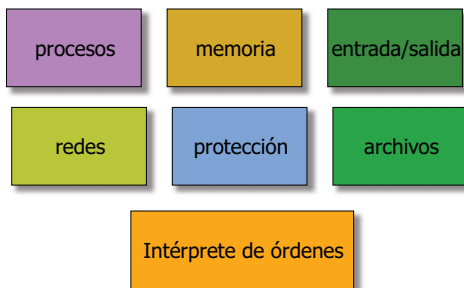
Contenidos

- Componentes típicos del SO
- Servicios del SO
- Llamadas al sistema
- Programas del sistema
- El núcleo o *kernel*
- Modelos de diseño del SO
- Cómo se implementa un SO



2

Componentes típicos de un SO



3

Gestión de procesos

procesos

- Un proceso es un programa en ejecución. Para poder ejecutarse, un proceso necesita tiempo de CPU, una porción de memoria, archivos, E/S y demás recursos.
- Responsabilidades del S.O.:
 - creación y eliminación de procesos
 - planificación de procesos: repartir la CPU entre los procesos activos
 - sincronización entre procesos
 - comunicación entre procesos



4

Gestión de memoria

memoria

- La memoria es un recurso escaso por el que compiten los distintos procesos.
- Responsabilidades del S.O.:
 - conocer qué zonas de memoria están libres y cuáles están ocupadas
 - decidir qué procesos hay que cargar cuando haya memoria libre
 - reservar y liberar zonas de memoria según se solicite
 - memoria virtual: utilizar el almacenamiento secundario como una extensión de la memoria principal.



5

Gestión de la E/S

entrada/salida

- La E/S es un conjunto de dispositivos muy variados y complejos de programar.
- Objetivos del S.O.:
 - proporcionar una interfaz uniforme para el acceso a los dispositivos (independencia del dispositivo)
 - proporcionar manejadores para los dispositivos concretos
 - tratar automáticamente los errores más típicos
 - para los dispositivos de almacenamiento, utilizar cachés
 - para los discos, planificar de forma óptima las peticiones



6

Sistema de archivos

archivos

- Un archivo es un conjunto de datos identificado por un nombre. Los archivos se almacenan en dispositivos de E/S. Un archivo es un concepto de alto nivel que no existe en el hardware.
- Funciones del S.O.:
 - manipulación de archivos: crear, borrar, leer, escribir...
 - manipulación de directorios
 - ubicar los archivos y directorios en los dispositivos de almacenamiento secundario
 - automatizar ciertos servicios: copia de seguridad, versiones, etc.

Sistema de protección

protección

- La protección abarca los mecanismos destinados a controlar el acceso de los usuarios a los recursos, de acuerdo con los privilegios que se definan.
- Objetivos del S.O.:
 - definir el esquema general de protección: clases de usuarios, clases de permisos/privilegios, etc.
 - definir mecanismos de acceso a los recursos: contraseñas, llaves, capacidades, etc.
 - controlar el acceso a los recursos, denegando el acceso cuando no esté permitido

Redes

redes

- En un sistema distribuido, existen varios ordenadores con sus propios recursos locales (memoria, archivos, etc.), conectados mediante una red.
- Objetivos del S.O.:
 - proporcionar primitivas para conectarse con equipos remotos y acceder de forma controlada a sus recursos: primitivas de comunicación (enviar y recibir datos) sistema de ficheros en red (ej. NFS) llamada remota a procedimiento (RPC) etc.

Intérprete de órdenes (command interpreter)

Intérprete de órdenes

- Para que un usuario pueda dialogar directamente con el S.O., se proporciona una interfaz de usuario básica para:
 - cargar programas
 - abortar programas
 - introducir datos a los programas
 - trabajar con archivos
 - trabajar con redes
- Ejemplos: JCL en sistemas por lotes, COMMAND.COM en MS-DOS, **shell** en UNIX

Servicios del SO

- El S.O. ofrece a los programas una serie de servicios para trabajar en el computador:
 - Ejecución de programas
 - Operaciones de E/S
 - Manipulación de archivos y directorios
 - Comunicación entre procesos
 - Comunicación con equipos remotos
 - Administración de la protección y seguridad
 - Leer el estado del sistema (hora, nº de procesos, etc.)

Servicios adicionales

- Aparte de los servicios básicos, el S.O. puede ofrecer algunas funciones para optimizar el uso del sistema:
- Compartición de recursos
- Contabilidad (*accounting*) - conocer el consumo de recursos

Interfaces con los servicios del SO

- Para el programador:
 - LLAMADAS AL SISTEMA en lenguaje máquina o en alto nivel (ej. lenguaje C)
- Para el usuario:
 - intérprete de órdenes
 - programas del sistema

¿Qué aspecto tiene una llamada al sistema?

- Windows:
 - `handle = OpenFile("mifichero", ofstruct, OF_READ)`
- UNIX:
 - `fd = open("mifichero", O_RDONLY);`
- MSDOS:

```
mov ah, servicio
mov al, modo
lea es:cx, cadena
int 21h
```

Llamadas al sistema

- El S.O. ofrece una gama de servicios a los programas.
- Los programas acceden a estos servicios mediante llamadas al sistema.
- Las llamadas al sistema son la interfaz entre el programa en ejecución y el S.O.
- Es la única forma en la que un programa puede solicitar operaciones al S.O.

Implementación de las llamadas al sistema

- ¿Cómo se implementa la llamada?
 - Habitualmente, mediante una instrucción especial de la máquina (`syscall`, `int`, `trap...`)
 - La instrucción cambia automáticamente a modo privilegiado
 - Si programamos en un lenguaje de alto nivel, escribimos la llamada al sistema como una subrutina, y el compilador la sustituye por la instrucción de máquina correspondiente.

Implementación de las llamadas al sistema (2)

- Muchas llamadas necesitan parámetros, ¿cómo los pasamos al S.O.?
 - guardándolos en registros de la máquina (muy típico)
 - en una tabla en memoria principal
 - poniéndolos en la pila

Ejemplo: llamadas al sistema de Unix

- Procesos: crear proceso (`fork`), enviar señal a un proceso (`kill`), tratar señales (`signal`)...
- Memoria: pedir más memoria, liberar memoria...
- Archivos: `open`, `close`, `creat`, `read`, `write`, `mkdir`; bloquear fichero (`lockf`)...
- Redes: crear conexión (`socket`), cerrar conexión...
- Protección: cambiar permisos (`chmod`), cambiar propietario (`chown`)...

Programas del sistema

- Las llamadas al sistema nos proporcionan una interfaz para el programador. Un usuario final interactúa con el S.O. mediante programas previamente compilados.
- El entorno del S.O. Suele proveer utilidades básicas, llamadas programas del sistema, para:
 - manipular ficheros (ej. ls, cp, etc.)
 - editar documentos (emacs, edit, etc.)
 - darnos un entorno de trabajo (ej. escritorio Windows)
 - desarrollar programas (compiladores, enlazadores, etc.)
 - comunicarnos con otros equipos (telnet, ftp, etc.)

El núcleo (kernel)

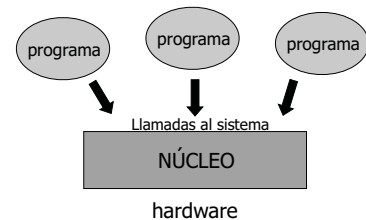
- Se suele llamar núcleo (kernel) al software del sistema operativo que reside permanentemente en memoria y que atiende las llamadas al sistema y demás eventos básicos.
- El núcleo se distingue de los programas del sistema (que utilizan los servicios del núcleo).

Modelos de diseño de un SO

- ¿Cómo está construido por dentro un sistema operativo?
- Hay múltiples técnicas:
 - Diseño **monolítico**
 - Diseño en **capas**
 - **máquinas virtuales**
 - Modelo **cliente-servidor**
 - **Micronúcleos**
 - ...

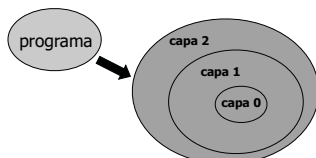
Diseño monolítico

- La arquitectura más simple para un S.O. Es un núcleo compacto, que contiene todas las rutinas de S.O.



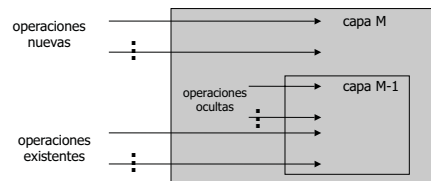
Diseño por capas

- El S.O. se construye en niveles jerárquicos, cada uno de los cuales aprovecha los servicios del nivel inferior.
- Diseño más modular y escalable que el monolítico.



Diseño por capas (2)

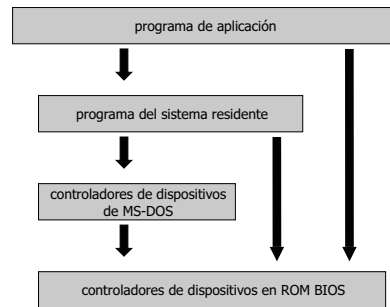
- Cada capa del SO consistiría en la implementación de un objeto abstracto
 - Datos
 - Operaciones



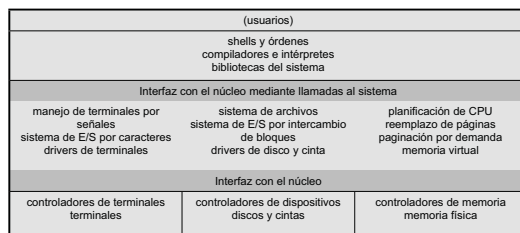
Ejemplo por capas: THE

- Sistema experimental de los años 60
- Seis niveles:
 - L5: aplicaciones de usuario
 - L4: buffering
 - L3: consola del operador
 - L2: gestión de memoria paginada
 - L1: planificación de procesos
 - L0: hardware

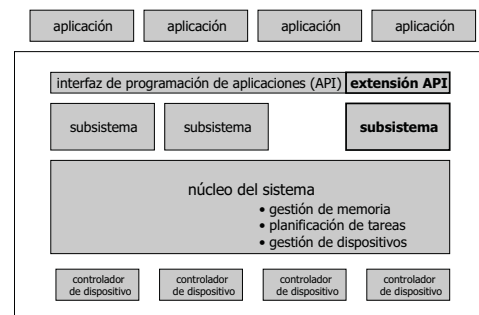
Ejemplo: MS-DOS (Microsoft)



Ejemplo: Unix (AT&T)



Ejemplo: OS/2 (IBM)



Diseño por capas: ventajas sobre monolítico

- La modularidad simplifica:
 - Depuración y verificación
 - Una vez depurada la primera capa se puede dar por sentado su funcionamiento correcto mientras se trabaja con la segunda capa.
 - Mantenimiento
 - Es posible por ejemplo cambiar las rutinas de bajo nivel siempre que la interfaz externa de la rutina no cambie y la rutina realice la misma tarea anunciada

Diseño por capas: desventajas sobre monolítico

- Problema: definición apropiada de las distintas capas
- Tienden a ser menos eficientes
 - Llamadas entre capas => paso de parámetros
 - En definitiva cada capa implica un gasto extra
 - Tendencia: equilibrio, menos capas con más funcionalidad:
 - Ventajas de la modularidad
 - Evitan los problemas de definición e interacción entre capas

Máquinas virtuales

- **La idea:** mediante software, se proporciona a los programas la *emulación* de un sistema que nos interesa reproducir.
- El sistema emulado puede ser una máquina, un sistema operativo, una red de computadores...
- El software emulador traduce las peticiones hechas a la máquina virtual en operaciones sobre la máquina real.
- Se pueden ejecutar varias máquinas virtuales al mismo tiempo (ej. mediante tiempo compartido).
- Los recursos reales se reparten entre las distintas máquinas virtuales.

Máquinas virtuales: ejemplos

- **IBM VM:** ofrecía a cada usuario su propia máquina virtual no multiprogramada; las m.v. se planificaban con tiempo compartido.
- **Java:** los programas compilados en Java corren sobre una máquina virtual (JVM).
- **VMware:** capaz de ejecutar al mismo tiempo varias sesiones Windows, Linux, Mac OS X, etc. sobre plataforma PC o Mac.
- **Nachos:** S.O. que se ejecuta en una máquina virtual MIPS, cuyo emulador corre sobre Unix.

Máquinas virtuales: pros y contras

- Protección: cada máquina virtual está aislada de las otras y no puede interferir
- Independencia de la plataforma (ej. Java)
- Pervivencia de sistemas antiguos (ej. emuladores MSDOS)
- Experimentación: se puede desarrollar y ejecutar para un hardware que no tenemos
- **Pero...** el rendimiento de la m.v. puede ser muy lento

Modelo cliente-servidor

- Según este modelo, el SO se organiza como un conjunto de módulos autónomos, cada uno de los cuales tiene a disposición del resto una serie de servicios
- Cada módulo actúa como un **servidor** de ciertas funcionalidades, que atiende las peticiones de otros módulos y que su vez puede ser **cliente** de otros módulos

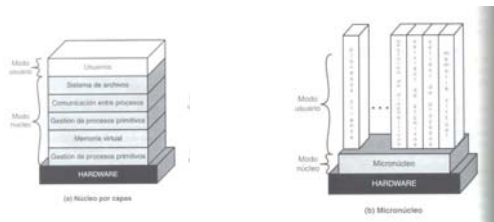
Modelo cliente-servidor

- Podemos extender el modelo cliente-servidor hasta el infinito, si consideramos cada módulo del sistema como un conjunto de módulos con relaciones cliente-servidor
- El modelo jerárquico no es más que un caso particular del modelo cliente-servidor
- Indicado para sistemas distribuidos

Micronúcleo

- Objetivo: construir un núcleo del SO con lo mínimo imprescindible
 - cambios de contexto, gestión de interrupciones, comunicación entre procesos, etc.
- Las políticas de gestión de los recursos se implementan **fuera** del núcleo, como procesos externos a nivel de usuario
- Primer micronúcleo: **Mach** (1980)

Micronúcleo (2)



Micronúcleos: ventajas

- Se pueden incorporar nuevos módulos sin necesidad de alterar el núcleo
- Se cargan en memoria sólo aquellos módulos que sean necesarios en cada momento
- Pueden aplicarse diferentes políticas para distintas clases de procesos (ej. unos por lotes, otros en tiempo real...)
- Puede servir como base de múltiples sistemas operativos (estos se implementan como colecciones de módulos cargables)
- Facilita la implementación de máquinas virtuales

Micronúcleos: ventajas (2)

- **Fiabilidad**
 - "Lo pequeño es bello"
 - el micronúcleo es menos complejo y por tanto más fácil de depurar
 - Muchos módulos se ejecutan como procesos de usuario
 - si un servicio falla, el resto del SO puede seguir adelante

Implementación de un sistema operativo

- Como todo software, debe seguirse un proceso de desarrollo (ingeniería de sw):
 - requisitos, diseño, construcción, pruebas, paso a explotación, mantenimiento...
- Pero un SO presenta características especiales:
 - Es un sistema crítico (todas las aplicaciones dependen de él)
 - Normalmente hay requisitos más estrictos de portabilidad (respetar versiones anteriores)
 - Es más complicado de depurar... ¿cómo probamos un pequeño cambio? ¿tenemos que volverlo a instalar un equipo?

Separar mecanismos y políticas

- Los mecanismos definen cómo se realiza algo; las políticas definen qué se debe realizar.
- Ejemplos:
 - Tiempo compartido → política
 - Temporizador, colas de procesos, gestión de interrupciones → mecanismos
- Es conveniente separar en el código los mecanismos de las políticas. Aunque la política cambie, los mismos mecanismos pueden seguir siendo útiles.

Lenguaje para implementar un SO

- En el pasado, lenguaje ensamblador (por eficiencia).
- Hoy día se usan lenguajes de alto nivel, sobre todo C/C++.
- Más legible y fácil de mantener y depurar
- Más transportable a distintas arquitecturas hardware